

1stOne F1 — Master Reference Document

Mobile App, Landing Page, and Web Application

2026-04-26

Table of Contents

1stOne F1 — Master Reference Document.....	1
1. Executive Summary	1
2. Workflows by Role.....	2
3. Business Logic — How Decisions Are Made.....	5
4. Edge Functions — The Backend Logic	8
5. Database — What Lives Where.....	9
6. Three Deployment Surfaces	10
7. Dependencies Table.....	11
8. Technology Stack Summary	15
9. Critical Points to Record	15
10. Where the Source Lives.....	18

1stOne F1 — Master Reference Document

Last updated: 26 April 2026 **Version:** 1.0 **Scope:** Mobile app (iOS + Android), landing page (1stone.in), and web application (app.1stone.in) **Owner:** Shrikanth Hegde

1. Executive Summary

1stOne is a meal and essentials delivery service operating in Siddapur, Uttara Kannada, Karnataka. The brand connects a home kitchen directly with subscribers — no middlemen, no factory food.

The product surface is split across three pieces, all sharing a single Supabase backend:

Surface	URL	Audience	Status
Mobile app	(App Store / Play	Customers, staff,	Working

Surface	URL	Audience	Status
	Store — pending submission)	admin	
Landing page	https://1stone.in	Public marketing + app discovery	Live
Web application	https://app.1stone.in	Admin + staff (full), customer (limited)	Live

The mobile app is the primary surface. Customers can browse menus, subscribe to a plan or place a one-off order, manage their delivery address, top up a wallet, and track orders end-to-end. Staff use the same app on a different role to run the kitchen, packing, and delivery flows. Admins manage everything else — plans, menu, hubs, zones, employees, banners, settings — through a unified two-tab interface (Reports and Manage). The web application mirrors the mobile experience for desktop use, with one deliberate restriction: any flow that needs Razorpay (the payment gateway) is blocked on web because Razorpay’s React Native software development kit does not run in browsers. Customers on web see a friendly “Mobile App Required” message in those moments.

The static landing page exists for two reasons: search engine visibility (Google indexes static HTML far better than single-page applications) and a clean entry point for new visitors. It carries the brand story, app download links (placeholders until apps are listed), an FAQ page, and a discrete admin/staff login link in the footer. The hero banner on the landing page is admin-editable from the same screen that manages the mobile login background — change the image once, both surfaces update on the next page load.

The system uses three user roles — customer, staff, admin — derived from a custom claim baked into the JSON Web Token (JWT) at login. A super-admin sees every branch; a regional admin or staff member is automatically scoped to their assigned branch. Identity is phone-OTP-based (no passwords).

2. Workflows by Role

This section explains what each role does in plain language. Every screen is functional today.

2.1 Customer Workflow

A customer signs in with their phone number and a six-digit one-time password (OTP). On first sign-up they enter their name and a delivery address; the system runs a serviceability check and either confirms coverage or notes the address as outside the service area.

After login the customer lands on the home screen, which shows the current day’s delivery cycles in two tabs — Meal and Essentials. Each cycle (e.g., Breakfast, Lunch, Snacks, Dinner) lists the menu items available for that cycle along with prices. A live banner section

shows promotional content set by admin. The user can tap any item to add it to their cart, or jump to the Plans screen to subscribe to a multi-day plan.

The cart is cycle-aware — items are grouped by which delivery cycle they belong to. There is also a **one-plan invariant** for subscriptions: a customer can put exactly one subscription plan in the cart at a time. Adding a second plan replaces the first, preventing accidental double-charges.

At checkout, the customer picks a delivery address (defaulting to the most recently used), chooses a payment method (wallet or online via Razorpay), and confirms the order. If they choose wallet and have enough balance, the order is created instantly and marked Confirmed. If they choose Razorpay, the system creates a payment order, opens the Razorpay sheet, and only marks the order Paid after Razorpay's webhook confirms the payment via cryptographic signature.

After placement, the order moves through eight statuses tracked in the database: Pending → Confirmed → Preparing → Ready → Packed → Dispatched → On the Way → Delivered. The customer sees these updates live, with push notifications at key milestones.

Subscription customers can pause their subscription with one tap (no deliveries until they resume), skip individual days (useful when traveling), and view a delivery calendar showing past, upcoming, and skipped days. Pauses and skips do not consume subscription days — the duration extends so every paid meal eventually gets delivered.

The wallet screen shows the current balance and a transaction history. Top-ups go through Razorpay; the wallet is credited only after the payment webhook fires (no race condition possible). A low-balance nudge appears when the balance falls below an admin-configured threshold.

The referral screen shows the customer's referral code, links to share, and a list of past referrals with their reward status. The system credits the referrer at three milestones — when the referee signs up, when they complete their first order, and when they finish their first month.

The feedback screen lets customers rate the app overall or rate individual items they received. Ratings flow to admin reports.

2.2 Staff Workflow

Staff sign in the same way customers do; the JWT custom claim routes them into the staff dashboard instead of the customer home. The staff dashboard is a single screen with three top tabs: Kitchen, Packing, Delivery.

Kitchen shows aggregated item counts for the next dispatch — for example, if 23 customers ordered Breakfast Idli, the kitchen sees “Idli × 23” rather than 23 individual order rows. This helps the cook prepare in batches. Staff can mark Confirmed orders as

Ready when food is plated. There are also footer buttons for raising supply requests for vegetables and grocery — these go to admin for approval.

Packing shows orders with full item lists, split into Food and Essentials sub-tabs. Staff toggle status from Ready to Packed as they assemble each order. Footer buttons can print labels or a summary sheet (using the device's print driver), and request stationery supplies.

Delivery shows Dispatched orders. Staff (typically the driver) update status from On the Way to Delivered as routes complete. A footer link opens a route map PDF stored centrally.

Staff also have screens for daily attendance (clock in / clock out, captured with GPS coordinates), leave requests (admin approves), and expense claims (admin approves and marks paid). A profile screen shows their designation, branch, and shift.

If admin has set a “note to staff” for the active tab, it appears as a yellow banner at the top of the dashboard — useful for one-off announcements like “Lunch dispatch delayed 30 minutes today.”

Staff visibility is hub-scoped: a staff member assigned to a particular delivery hub sees only the orders routed to that hub. This stays clean as the operation scales beyond one hub.

2.3 Admin Workflow

Admins land on AdminHome, which has two tabs: **Reports** and **Manage**.

Reports shows the day's headline metrics — revenue, order count, active subscriptions, staff present today, hub deliveries — each with a tap-through to a detail report. Detail reports support date range filters and CSV export. There are five detail reports: orders, revenue, subscriptions, staff, and hub delivery.

Manage is a settings-style list. Each row drills into a focused screen for one concern. The list is intentionally long but organized into logical groups (Marketing, Menu, Plans, Operations, People, Settings). Notable management screens:

- **Plans** — create, edit, deactivate subscription plans. Each plan defines its cycle, duration in days, price, and the list of items it delivers.
- **Menu** — manage daily menu items per cycle. Items can be activated for one cycle and deactivated for another (e.g., Idli only at Breakfast).
- **Essentials** — separate catalog for non-meal items (milk, bread, etc.) with a dedicated cycle.
- **Delivery Hubs** — define hub locations with polygon coverage, assigned staff and driver, optional fee override and commission percent.
- **Delivery Zones** — define service-area polygons for the base service area. Used by the serviceability check.
- **Branches** — multi-branch ready (most operations today are single-branch).

- **Employees** — onboard new staff (elevates a customer profile to staff role), edit existing staff details, view their attendance / leave / salary / expense history, assign a hub.
- **Banners & Backgrounds** — single screen with three sections in order: Special Offer Banners (drill-in to dedicated screen with its two tabs), Phone Login Background, Website Landing Banner. All admin-editable; mobile and web pick up new images on next launch / reload.
- **Notification Templates** — admin can edit the title and body of every push notification the system sends. Variables in the template (like the customer’s name or order number) get filled in at send time.
- **Feature Flags** — runtime toggles. Most prominent is “storm mode” — when on, all new orders are rejected (“Orders temporarily paused”). Useful for monsoon, technical issues, or supply outages.
- **Store Config** — global settings: delivery fee, tax rate, minimum wallet top-up, low-wallet threshold, loyalty points per rupee, cancellation window, and several module toggles.
- **Special Offers** — full management of promotional banners shown on the customer home screen, with two internal tabs (active vs. scheduled).
- **Referral Settings** — adjust all referral reward amounts and milestone thresholds.

If the admin has no `branch_id` in their JWT (a “super-admin”), they additionally see a branch selector at the top of the Manage tab — switching branches scopes all subsequent data filters to that branch.

3. Business Logic — How Decisions Are Made

This section captures the rules that govern the system. These are decisions, not technical implementations.

3.1 Time and Cycles

A delivery cycle has a cutoff time and a delivery start time. The system uses these to decide when an order ships:

- If the customer orders **before** the cutoff, the order ships in **today’s** cycle.
- If the customer orders **after** the cutoff, the order ships in **tomorrow’s** cycle.

The system also handles cross-midnight cycles. If an admin defines a cycle with cutoff 22:00 and delivery start 07:30, the cycle wraps around midnight. The logic correctly treats:

- Orders placed before 07:30 → today’s batch (it’s already past midnight from the previous day’s window).

- Orders placed between 07:30 and 22:00 → tomorrow's batch (we're now in tomorrow's ordering window).
- Orders placed after 22:00 → today's batch (the cutoff has passed, but the cycle continues).

All time comparisons use **server time only** — the device clock is never trusted for business decisions. This prevents users from changing their phone clock to get a different cycle.

3.2 Subscription Conflicts

Two subscriptions on the same cycle can conflict only if they share core item IDs — even if their plan names differ. Example: a “15-Day Bread” plan and a “30-Day Bread” plan both deliver bread item ID = 5. The system catches this overlap regardless of plan naming.

When a conflict is detected, the customer is shown a “Start After” dialog: the new plan can be queued to start the day after the existing one ends (`start_date + duration_days`). The customer can accept this or change the date manually.

Food and essentials plans never conflict with each other — they're treated as separate streams.

3.3 Serviceability — Where We Deliver

When a customer adds a delivery address, the system runs a three-step check:

1. **Zone match.** The latitude/longitude is checked against active delivery zones (polygons drawn by admin). If inside any zone, the address is serviceable.
2. **Hub extension.** If outside all zones, the system checks delivery hubs that have “`extends_coverage`” turned on. This is for outliers — say, an office cluster across a highway — that an admin wants to serve without redrawing the entire zone polygon.
3. **No match.** If neither matches, and the database has any zones or hubs configured, the address is marked not serviceable. If nothing is configured at all, it's marked unknown (preserves operability during initial setup).

The address's delivery fee comes from (in priority order): the hub's fee override, the zone's fee override, or the global store config default. The cheapest configurable option wins.

3.4 Hub-Based Delivery

Delivery is hub-routed. Each customer address is assigned to one delivery hub (based on which hub's polygon contains the address). Staff assigned to that hub see only those orders. A hub also has one designated driver (a staff member with a vehicle); the driver sees their hub's orders in the Delivery tab.

This means the operation can scale to many hubs without each staff member seeing every order in the city.

3.5 Wallet Decrement Atomicity

The wallet balance is never read-then-written from the client. Doing so would create a race condition (two simultaneous orders could both see “enough balance” and both succeed, going negative). Instead, every wallet debit is a single atomic database call: “if you have X rupees, deduct X rupees and tell me yes; otherwise tell me no, no change made.”

This guarantees the wallet can never go negative, regardless of how many orders fire at once.

3.6 Idempotency for Payments

Payment-related endpoints are idempotent. Every order placement, wallet top-up, or payment confirmation request must include a unique Idempotency-Key in the request header. If the same key is seen twice (e.g., the network dropped and the client retried), the cached response from the first request is returned — no second order is created, no second debit happens.

The same table is also used for rate limiting: maximum 5 calls per user per endpoint per 60 seconds.

3.7 Razorpay Webhook Idempotence

Razorpay can fire the payment.captured webhook multiple times for the same payment (network retries on their end, manual re-triggers, etc.). Every database operation triggered by the webhook checks “is this already done?” before doing it. The system reaches the Paid state exactly once, no matter how many duplicate webhooks arrive.

3.8 Storm Mode (Kill Switch)

Storm mode is a single toggle that immediately stops accepting new orders. Useful during monsoon, kitchen incidents, supply chain breaks. There are two locations admin can toggle it: store config (a single column) and feature flags (a runtime row). Either being true stops orders. This dual-control gives admins flexibility — feature flags can be flipped without regenerating any cached state.

3.9 Notification Template System

Every push notification the system sends is dispatched through a template lookup. Admin can edit the title and body of any notification — for example, change “Order Confirmed!” to “Got it! Your order is confirmed” — without any code change. Templates use placeholder variables (like {{order_number}}) that get filled at send time. If a template is disabled, that event silently doesn’t fire. If the templates table is missing or empty, every send falls back to a hardcoded default — the system stays operational.

Every push attempt is logged for audit (who got what, when, success or failure).

3.10 Branch Filtering

Branches are a multi-tenancy mechanism. A super-admin (no `branch_id` in JWT) sees every branch's data. A branch-scoped admin or staff member is locked to their branch by row-level security policies in the database — they can't even read another branch's orders, let alone modify them. Customers don't have a `branch_id` in their JWT; their branch is implicit through the address they deliver to.

3.11 Loyalty and Referrals

Customers earn loyalty points proportional to their order value (rate set in store config). Points can be redeemed for credits. The transaction log is append-only (the running balance is on the profile).

Referrals work via an admin-configurable rules engine: each referee signup, first order, and first month milestone triggers a configurable credit (rupees and/or points) to either or both parties. The rule sheet is edited in the Referral Settings screen.

3.12 Server-Side Authority for Money

The client is **never trusted** with anything money-related:

- Item prices are validated server-side at order placement (the client's display price is informational only).
- Delivery method (hub or standard) is derived server-side from the address — the client cannot fake it.
- Delivery fee is computed server-side from the address's hub/zone/store config.
- Subscription conflicts are re-checked server-side.

This protects against tampering. The client UI shows the user what they will be charged, but the server is the only authority on what they actually pay.

4. Edge Functions — The Backend Logic

The system runs 11 edge functions — small pieces of server code that handle specific tasks. Each one does one thing and does it idempotently.

Function	What it does
place-order	Places a customer order (food, essentials, or subscription). Validates items, derives delivery method server-side, enforces storm mode, calls the atomic insert. Required to use Idempotency-Key header.
verify-payment	Razorpay webhook handler. Verifies the

Function	What it does
	cryptographic signature, then marks the relevant record as Paid (order, wallet topup, or subscription). Idempotent.
confirm-order	Manual fallback if the webhook is delayed. Customer-side confirms the payment. Verifies signature before doing anything.
confirm-topup	Same as confirm-order but for wallet top-ups.
wallet-topup	Creates a Razorpay payment order for wallet top-up. Stores a pending record; the actual wallet credit happens only when the webhook fires (never client-side).
send-push	Generic push dispatcher. Looks up Expo push tokens for a list of users (or a role filter), fans out to Expo's push API, logs every attempt.
subscription-expiry-push	Scheduled. Finds subscriptions ending soon, sends a "renewal reminder" push.
low-wallet-check	Scheduled. Finds users below the low-balance threshold, sends a top-up nudge.
dormant-user-check	Scheduled. Finds users who haven't ordered in N days, sends a "we miss you" message.
apply-referral	Activates a referral code and credits both parties per the configured rules.
elevate-employee	Promotes an existing customer profile to staff with all the staff-specific fields (designation, salary, hub assignment, joining bonus).
cancel-order	Customer-initiated cancellation within the cancellation window. Refunds wallet or initiates Razorpay refund, marks order Cancelled.

5. Database — What Lives Where

The Supabase database has approximately 35 tables. Rather than listing every column, here's what each group does:

Identity: Profiles (one row per user) and branches.

Catalog: Menu items, essentials catalog, delivery cycles, subscription plans, plan items.

Customer-owned: Customer addresses, orders, order items, user subscriptions, cancelled subscription days, wallet transactions, pending wallet topups, loyalty redemptions, referrals, app feedback, order item ratings, push notification tokens.

Operations: Delivery hubs, delivery zones, kitchen push log, manifest run log, banners.

Staff-owned: Staff attendance, staff leaves, staff salary, staff shifts, expense claims, staff order requests, supply catalog, supply batches, supply order items.

Configuration: Store config (single row of global settings), feature flags (runtime toggles), app settings (URLs for hero images), notification templates, referral settings, admin notes (per-tab messages for staff dashboard), business expenses, idempotency keys, push logs.

Every table that users can write to has Row-Level Security policies enforcing scope: customers see only their own data, staff see only their hub or branch, admins see only their branch (or everything if super-admin). The recent audit added “WITH CHECK” clauses everywhere they were missing — so a user can’t, for example, change the `user_id` on their own row to point at someone else’s data.

6. Three Deployment Surfaces

6.1 Mobile App (iOS + Android)

Built with Expo. Distributed through Expo Application Services (EAS) — a managed build pipeline that produces production iOS and Android binaries from the same JavaScript codebase. Released to the App Store and Play Store (pending publication as of this document). Push notifications go through Expo’s notification service, which is a managed wrapper around Apple’s APNs and Google’s FCM.

6.2 Landing Page (1stone.in)

Pure static HTML and CSS in a `landing/` folder of the repository. Three files: `index.html` (the homepage), `faq.html` (FAQ), `styles.css` (shared styling), plus `robots.txt` and `sitemap.xml` for search engines. Deployed by Cloudflare Pages from the GitHub repository — every push to the main branch triggers an automatic redeploy. The DNS for `1stone.in` is managed by Cloudflare; the GoDaddy nameservers were updated to Cloudflare’s during setup. The `www` subdomain redirects to the apex.

6.3 Web Application (app.1stone.in)

Built with Expo Web (React Native Web). Same codebase as the mobile app, compiled to a JavaScript bundle (~4.6 MB raw, ~1.2 MB after gzip compression). Deployed by Cloudflare Pages from the same repository, with the build command `npx expo export -p web`. All three roles can sign in. Customers on web can browse, view their account, and place

orders that pay from wallet — but Razorpay flows (online payments and wallet top-ups) are deliberately gated with a “Mobile App Required” message, because Razorpay’s React Native SDK does not support browsers.

The cryptographic safety of payments is unaffected — the gate is at the user interface layer; the server-side payment safety net (HMAC verification on the webhook) is identical.

7. Dependencies Table

This is the complete list of external services and software libraries the project depends on. The “Why” column explains why each is needed.

7.1 External Services and Platforms

Service	Why we use it	Credentials needed	Your notes
GitHub	Source code hosting; trigger for both Cloudflare deployments.	GitHub account login	<i>to be filled</i>
Supabase	Database (PostgreSQL), authentication (phone OTP), file storage (logo, banners, PDFs), edge functions (server-side business logic), real-time subscriptions. The single biggest backend dependency.	Project URL, anon key, service role key, dashboard login	<i>to be filled</i>
Cloudflare	DNS, free static hosting (both 1stone.in and app.1stone.in), HTTPS / SSL, www-to-apex redirect.	Cloudflare account login	<i>to be filled</i>
GoDaddy	Domain registrar for 1stone.in. (DNS itself runs on Cloudflare.)	GoDaddy account login	<i>to be filled</i>

Service	Why we use it	Credentials needed	Your notes
Expo / EAS	Mobile app build pipeline (iOS, Android), over-the-air updates, push notification service.	Expo account login, EAS project ID	<i>to be filled</i>
Razorpay	Payment gateway. Handles UPI, cards, net banking. Credit/debit cards stored on Razorpay's servers (PCI-compliant), never on ours.	Key ID (public), Key Secret, Webhook Secret, dashboard login	<i>to be filled</i>
MSG91	SMS gateway for OTP delivery. Replaces Supabase's default Twilio-based provider — cheaper and India-optimized. (Pending account activation as of this document.)	Auth key, sender ID	<i>to be filled</i>
Sentry	Error tracking and crash reporting in production. Tags every error with the user ID for traceability.	DSN (data source name), dashboard login	<i>to be filled</i>
Apple Developer Program	Required to publish to the App Store. (\$99/year.)	Apple ID, team ID	<i>to be filled</i>
Google Play Console	Required to publish to the Play Store. (\$25 one-time.)	Google account, console login	<i>to be filled</i>

7.2 Optional / Future

Service	Why we may use it	Status
PostHog	Product analytics — understand which screens users visit, where they drop	Library installed, not yet wired

Service	Why we may use it	Status
Google Maps	off. Already used for drawing delivery zones. The Google Maps API key is in the codebase but only consumed within the admin's zone-drawing screen.	Working
Firebase	Not directly used. Expo Push internally uses FCM (Firebase Cloud Messaging) for Android delivery, but we do not configure Firebase ourselves — Expo handles it.	Indirect via Expo

7.3 Software Libraries (mobile + web bundle)

Library	Version	Purpose
Expo	~54.0.0	The framework that wraps React Native with build tooling, push notifications, file system access, image picker, and dozens of small utilities.
React Native	0.81.5	The cross-platform UI framework that lets one codebase build for iOS, Android, and web.
React	19.1.0	The component model that React Native is built on.
React DOM	^19.1.0	Required for the web build.
React Native Web	^0.21.2	The bridge that translates React Native components into web HTML.
TypeScript	~5.9.2	Static type checking — catches bugs before they reach production.
Supabase JS	^2.45.0	Official client library for talking to Supabase.
TanStack React Query	^5.60.0	Manages server state — caches API responses,

Library	Version	Purpose
		refetches when data changes, retries on failure.
Zustand	^5.0.0	Lightweight state management for cart, UI state, and offline mutation queue.
React Navigation	^7.0.0	Routing between screens.
AsyncStorage	2.2.0	Persistent local storage on mobile (cart survives app restart).
React Native Razorpay	^2.3.0	Razorpay's mobile SDK.
Sentry React Native	^8.7.0	Sentry SDK for crash reporting.
Expo Notifications	~0.32.16	Push notification primitives (token registration, foreground handler).
Expo Image Picker	~17.0.10	For admin to pick images (login background, landing banner).
Expo Location	~19.0.8	GPS for staff attendance, customer address geocoding.
Expo File System / Print / Sharing	~19, ~15, ~14	For admin reports CSV download and staff label printing.
React Native Maps	1.20.1	Native map component for iOS / Android (zone polygon drawing).
@react-google-maps/api	^2.20.8	Google Maps wrapper for the web build (zones admin only).
React Native Reanimated / Worklets	~4.1, 0.5	Animation primitives.
Jest / Jest-Expo / Testing Library	^30, ~54, ^5.4	Test runner and helpers. 191 unit tests pass.
Knip	^6.6.3	Dead-code detector (used during cleanup; runs on demand).
Patch-Package	^8.0.1	Lets us patch third-party

Library	Version	Purpose
libraries without forking them.		

8. Technology Stack Summary

Layer	Technology
Mobile platform	iOS 13+, Android 6+
Mobile framework	React Native 0.81.5 via Expo SDK 54
Language	TypeScript 5.9 (strict mode + no-unused-locals enforced)
Build (mobile)	Expo Application Services (EAS)
Web framework	React Native Web 0.21
Web hosting	Cloudflare Pages (free tier)
DNS + CDN	Cloudflare
Domain registrar	GoDaddy
Backend	Supabase (managed PostgreSQL + auth + storage + edge functions)
Database	PostgreSQL 15 (managed by Supabase)
Server-side language	TypeScript on Deno (Supabase Edge Functions)
Authentication	Phone OTP via Supabase Auth (planned move to MSG91 SMS provider)
Payments	Razorpay (test keys today; production keys before launch)
Push notifications	Expo Push Service (wraps APNs and FCM)
Error tracking	Sentry
Source control	Git, hosted on GitHub (shrikanth2018-stack/F1)
Continuous deployment	Cloudflare Pages auto-deploys on push to main branch
Test framework	Jest with jest-expo preset, 191 tests across 9 files

9. Critical Points to Record

These are non-obvious facts that future developers, ops teams, or you-in-six-months should know.

9.1 Phone Numbers Use E.164 Format Internally

All phone numbers in the system are stored as +91XXXXXXXXXX (the international E.164 format). The login and registration flows accept 10-digit Indian numbers and normalize them to this format before sending to Supabase. Any external system that integrates with the database (analytics dashboards, CRM imports) must use the same format or users won't be found.

9.2 The Custom Access Token Hook is Critical to Login

Login depends on a Postgres function called `custom_access_token_hook` that reads the user's profile (role, branch ID, assigned hub ID) and embeds those values in their JWT. If this function is disabled or its grant is revoked, every login will silently default users to "customer" role — breaking admin and staff access. There is a row-level security policy specifically for this hook so it can read the profiles table even when RLS is otherwise restrictive.

9.3 The Web Build Cannot Process Razorpay Payments

This is by design. Razorpay's React Native SDK is mobile-only; their web SDK is a different product not integrated into this codebase. The web user interface deliberately blocks payment-initiating actions on web with a friendly message pointing customers to the mobile app. Wallet payments (where wallet balance is sufficient and no Razorpay call is needed) still work on web.

9.4 The Service-Role Key Was Rotated and Scrubbed from History

A previous version of one SQL file contained the Supabase service-role key in plaintext. This was rotated to a new key, the old key was revoked, and the entire git history was rewritten to remove every trace. The old key was never pushed to GitHub (rotation was preventive — caught before first push). The new key is stored only in Supabase's `app_config` table for use by `pg_cron` and `pg_net`, never in the repository.

9.5 RLS Policies Use `WITH CHECK` Everywhere They Need To

A recent audit found 8 tables where RLS policies had a `USING` clause (read filter) but no `WITH CHECK` clause (write filter). This allowed an authenticated user to, for example, `INSERT` a row with another user's `user_id`. All 8 were fixed: `customer_addresses`, `expense_claims`, `push_notification_tokens`, `staff_attendance`, `staff_leaves`, `staff_salary`, `user_subscriptions`, `cancelled_subscription_days`.

9.6 React Native Web's `Alert.alert()` is Unreliable for Multi-Button Dialogs

The standard `Alert.alert()` function from React Native does not render correctly on React Native Web for confirmations with destructive buttons. The system uses a small wrapper called `confirmDialog` (and `infoDialog` for single-button alerts) that uses the browser's native `window.confirm()` / `window.alert()` on web and `Alert.alert()` on

mobile. Logout, payment-blocked alerts, and other multi-choice dialogs all go through this wrapper.

9.7 The Razorpay Webhook Must Be Verified

The webhook URL is publicly accessible. Without HMAC-SHA256 signature verification, anyone could POST to it and trigger fake payment confirmations. Every webhook hit is verified using the shared secret stored as the `RAZORPAY_WEBHOOK_SECRET` environment variable. If this secret leaks or is removed, the webhook will refuse all incoming requests until it's restored.

9.8 Push Notifications Don't Work on Simulators

The Expo push service requires a real iOS or Android device. Simulators and emulators cannot register push tokens. Testing the push pipeline end-to-end requires installing the app on a physical device. (The pipeline can be tested partially from SQL by calling the `send-push` edge function directly with a real device's token.)

9.9 Idempotency Keys Are Required for Payment Endpoints

The mobile and web clients automatically attach a unique Idempotency-Key header to every order placement, wallet top-up, and confirmation request. If a third-party tool or test script integrates with these endpoints and forgets the header, duplicate orders may result on network retries. The header is the only protection against this.

9.10 The Subscribe and Confirm-Subscription Edge Functions Are Deleted

These were legacy functions from an earlier subscription flow. They've been removed from the repository and from Supabase. Subscription activation now happens through `place-order` (creates the subscription) and `verify-payment` (activates it on payment success).

9.11 Sentry Is on a 14-Day Trial

Sentry was set up with a Business trial that auto-started at signup. Switch to the free Developer plan (5,000 errors/month, sufficient for current scale) before the trial ends to avoid an unexpected charge. No credit card was confirmed during signup, so worst case the trial expires harmlessly and the account auto-drops to the free tier.

9.12 The Banner / Background System Has a Single Source of Truth

The mobile app's login background, the website's hero banner, and the in-app special offer banners are all managed from one admin screen ("Banners & Backgrounds"). Each banner type has its own database column and its own image file in Supabase Storage. Replacing an image automatically updates everywhere it's referenced — mobile pulls fresh on next app launch, web pulls on next page reload, no rebuild needed.

9.13 No Lazy Loading on the Web Build

The 4.6 MB JavaScript bundle includes all customer, staff, and admin screens. After first load it's cached aggressively and subsequent visits are instant. Code-splitting by role would reduce first-load time but adds complexity; defer until real users complain about slow first load.

9.14 22 Transitive Vulnerabilities in npm audit

A run of `npm audit` reports 22 vulnerabilities (1 high, 17 moderate, 4 low). All are in transitive dependencies of the Expo SDK (postcss, xmldom, uuid, etc.). None affect 1stOne's specific code paths — they're "library has a known issue, but only triggers in usage patterns we don't use." Don't run `npm audit fix --force` — it would downgrade Expo and break the build. These will resolve when Expo SDK is upgraded to the next major version.

9.15 Cycle Definitions Drive Almost Everything

A delivery cycle (Breakfast, Lunch, etc.) is the central concept. Menus belong to cycles, plans belong to cycles, orders belong to cycles. Adding a new cycle (say, "Late Night Snacks") is a deliberate operation: it requires defining cutoff time, delivery start, kitchen push time, and `is_essentials` flag. The cycle name is what customers see; the schedule columns are the operational reality.

9.16 Edge Functions Always Return HTTP 200

Webhook endpoints (verify-payment in particular) always return HTTP 200, even on internal errors. Returning a 500 would cause Razorpay to retry indefinitely — but our errors are typically already-paid (idempotent) or schema-related (won't be fixed by retrying). Errors are logged to Sentry instead. The 200 means "received and acknowledged"; whether anything further was done is recorded in our own logs.

10. Where the Source Lives

Surface	Location
GitHub repository	https://github.com/shrikanth2018-stack/F1
Mobile app source	<code>src/</code> folder, builds via <code>npx expo start</code> and EAS
Landing page source	<code>landing/</code> folder (HTML/CSS)
Edge function source	<code>supabase/functions/</code> folder
Database SQL migrations	<code>supabase/sql/</code> and root <code>supabase/</code> folder
Master document (this file)	<code>MASTER_DOCUMENT.md</code> at repository root

End of document.